DIVERSITY EMPOWERS INTELLIGENCE: INTEGRATING EXPERTISE OF SOFTWARE ENGINEERING AGENTS

Kexun Zhang[1,2], **Weiran Yao**[1], **Zuxin Liu**[1], **Yihao Feng**[1], **Zhiwei Liu**[1], **Rithesh Murthy**[1], **Tian Lan**[1], **Lei Li**[2], **Renze Lou**[1], **Jiacheng Xu**[1], **Bo Pang**[1], **Yingbo Zhou**[1], **Shelby Heinecke**[1], **Silvio Savarese**[1], **Huan Wang**[1], **Caiming Xiong**[1]

[1]Salesforce AI Research, [2]Carnegie Mellon University

arXiv:2408.07060v1 [cs.SE] 13 Aug 2024

ABSTRACT

Large language model (LLM) agents have shown great potential in solving real-world software engineering (SWE) problems. The most advanced open-source SWE agent can resolve over 27% of real GitHub issues in SWE-Bench Lite. However, these sophisticated agent frameworks exhibit varying strengths, excelling in certain tasks while underperforming in others. To fully harness the diversity of these agents, we propose **DEI** (Diversity Empowered Intelligence), a framework that leverages their unique expertise. DEI functions as a meta-module atop existing SWE agent frameworks, managing agent collectives for enhanced problem-solving. Experimental results show that a DEI-guided committee of agents is able to surpass the best individual agent's performance by a large margin. For instance, a group of open-source SWE agents, with a maximum individual resolve rate of 27.3% on SWE-Bench Lite, can achieve a 34.3% resolve rate with DEI, making a 25% improvement and beating most closed-source solutions. Our best-performing group excels with a 55% resolve rate, *securing the highest ranking* on SWE-Bench Lite. Our findings contribute to the growing body of research on collaborative AI systems and their potential to solve complex software engineering challenges.

1 INTRODUCTION

Recent advancements in large language models (LLMs) have transformed software engineering (SWE) and other domains. Originally developed as chatbots (Schulman et al., 2022; Team, 2024), LLMs have evolved into the core of AI agents, capable of understanding and generating human-like conversations, as well as autonomously executing actions in both real-world and digital environments. SWE agents, a specialized subset of these AI agents, integrate these capabilities with software engineering tools and techniques for tasks like code generation, automated testing, and project management, aiming to identify and resolve practical software issues (Zhang et al., 2024).

In this paper, we study one specific task of SWE agents – resolving real-world GitHub issues based on their descriptions. Automatically fixing a bug in a code repository is an extremely challenging task that involves navigating extensive codebases, understanding complex function interactions, detecting subtle errors, and generating the correct fix patch. The large action space of SWE agents, together with long trajectories, inevitably result in the diversity of Github issue solutions, as shown in Figure 1. We have observed that different SWE agents resolve very different sets of issues (the colored girds in Figure 1**a**), despite having similar resolve rates (Figure 1**b**). This is probably due to different skill sets of SWE agents. For instance, OpenDevin (Wang et al., 2024c) explicitly instructs the LLM to first replicate the bug in an issue and executes its replication in a development workspace to provide feedback for its generated patches, but other agents like Moatless Tools (Örwall, 2024) and Agentless (Örwall, 2024) do not actually execute code in the issue-specific repository.

A garden's beauty never lies in one flower. Diversity in all its forms is the path to greatness.

*Code, data and leaderboard results at: `salesforce-research-dei-agents.github.io`

†Contact: `kexun@cmu.edu`, `weiran.yao@salesforce.com`

Similarly, the trend in the SWE agent community reflects this diversity—no single agent framework dominates in all capabilities. It is the flourishing variety within this community that sparks new ideas and leads to the development of better agents.

The variety in SWE agent capabilities inspires us to develop DEI, **D**iversity **E**mpowered **I**ntelligence, a framework that leverages the strengths of diverse agents. DEI aims to harness these varied skills to tackle a broader range of problems more effectively with a multi-agent ensemble system and a re-ranking pipeline, as showcased in Figure 1**c**. DEI functions as a meta-module that can be integrated with any existing agent framework, enabling scalable management and collaboration among agents to form a more powerful multi-agent software engineering organization.

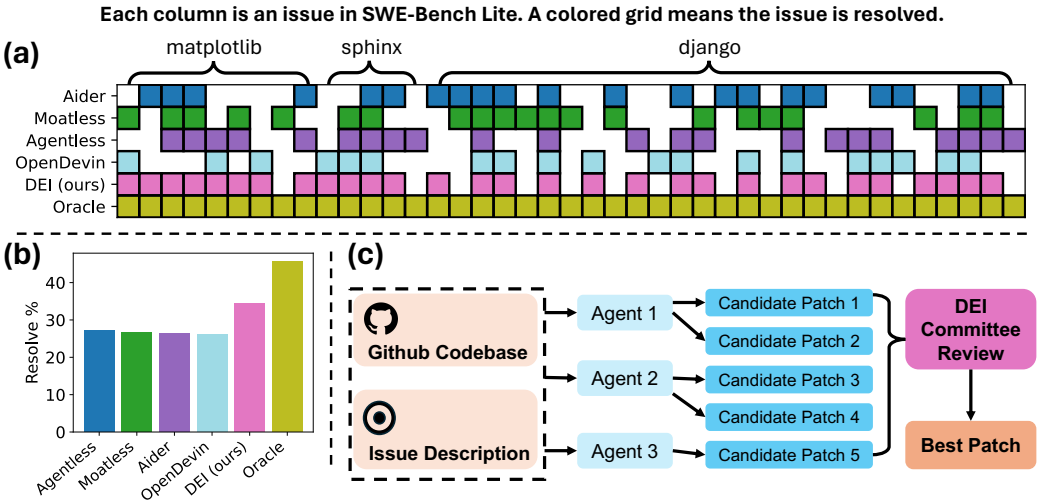

Figure 1: Different SWE agents (Aider, Moatless, Agentless, OpenDevin) resolve very different sets of issues (the colored girds in Fig **1a**), despite having similar resolve rates (Fig **1b**). Our proposed DEI Committee takes candidates patches and tries to select the best, oracle choice (Fig **1c**), improving the resolve rate significantly to be better than any single agent in the committee.

We evaluate DEI on 7 groups of candidate agents on SWE-Bench Lite. 3 of the 7 are different runs of a single open-source SWE agent. The other 4 are different agents that are on the SWE-Bench Lite leaderboard, including a group that contains only open-source agents. Through experiments, we find that different agents show a great level of diversity in the issues they resolve: a group of agents with an average resolve rate of 26.6% can collectively solve 54.3% of issues if there is an oracle reviewer that can consistently select the best candidate. DEI, as a first step towards harnessing such diversity, can improve the group's resolve rate to 34.3% ($\uparrow$ 25%), suggesting that LLMs are great code reviewers. These findings mirror the benefits of diversity in the tech industry, where diverse perspectives and skills lead to greater innovation and problem-solving capabilities.

To summarize, our contributions are the following:

- For the first time, we comprehensively evaluate the diversity of solutions provided by SWE agents, revealing significant differences in the types of GitHub issues resolved by various agents, despite similar overall resolve rates. These findings suggest a substantial potential to improve overall performance by more effectively leveraging the diverse expertise of these agents together.
- This paper introduces DEI, a multi-agent meta-policy module designed to harness the diversity of SWE agents and seamlessly facilitate collaboration among agents with different specialties. By employing a multi-stage rating and re-ranking pipeline, DEI consistently improves issue resolution, demonstrating a 25% performance boost on the SWE-Bench Lite leaderboard.

2 Related Work

We review the work in fundamental language agent architecture, recent developments for SWE agents, and multi-agent or ensemble methods in this section.

Fundamental Language Agent Pioneering AI agent methods along this line of work include ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), CodeAct (Wang et al., 2024b), etc., in which ReAct interprets the user query, generates functional API calls, and gets the tool outputs in real time; Reflexion further appends failed trial experience into the memory, enabling effective retrials to prevent repetitive errors. CodeAct (Wang et al., 2024b), instead of generating function calls, uses code generation to consolidate AI agents' actions into a unified action space.

Software Engineering Agent We present the SWE agents which have disclosed the technical details on the SWE-bench Lite leaderboard. Alibaba Lingma Agent (Ma et al., 2024) constructs a repository knowledge graph to represent code and dependencies, using a Monte Carlo tree search-based strategy for repository exploration, and generates patches to address real-world GitHub issues. AutoCodeRover (Zhang et al., 2024) adds advanced code search tools, such as abstract syntax trees, and spectrum-based fault localization to the agent for enhancing context understanding and issue resolution. Code-R (Chen et al., 2024) chooses a multi-agent framework with pre-defined task graphs to resolve Github issues. Agentless (Xia et al., 2024), is a simplified two-phase approach for solving software development problems. It focused on localization and repair without relying on LLMs to make decisions, highlighting the potential of straightforward techniques in autonomous software development. OpenDevin (Wang et al., 2024c) is a hub of community-contributed agents including CodeAct (Wang et al., 2024b), browser agent, GPTSwarm (Zhuge et al., 2024), and task-specific micro agents. Finally, SWE-agent (Yang et al., 2024) developed agent-computer interface that consists of LM-friendly commands and environment feedback to enable LM agents to autonomously use computers to solve software engineering tasks.

Multi and Ensemble Agents Recent works observe that organizing multiple specialized AI agents (Hong et al., 2024; Li et al., 2023; Liu et al., 2024) enable the task decomposition ability of an agent system, which improves the task-resolving performance. Current multi-agent frameworks are categorized into three types based on their execution patterns. Firstly, static agent working flow (Wu et al., 2024; Github, 2023), which pre-defines the agent execution flows and ignites agent transitions via specified conditions. Controlling a multi-agent system with pre-determined states is robust, though losing flexibility in terms of unseen states or conditions. Secondly, ensemble via group chatting (Wu et al., 2023; Hong et al., 2024; Wang et al., 2024a; Chen et al., 2023). This is built upon an environment where multiple agents send messages to each other in a group channel such that their thoughts are ensembled. Variants of group chatting includes debating (Liang et al., 2023; Chan et al., 2023) and model-wise ensembling (Wang et al., 2024a). Last but not least, hierarchical task assignment (Liu et al., 2024; 2023). Organizing multi-agent in a hierarchical structure benefits the top-down task decomposition and thus enables efficient multi-agent collaboration.

3 Integrating Expertise of SWE Agents

3.1 Background

Resolving issues in SWE-Bench. One important task in software engineering is to resolve issues raised by developers. SWE-Bench curates instances of this task by collecting successfully resolved issues from open-source repositories on Github. Each instance in SWE-Bench consists of a textual issue description, a version of the repo just before the issue was resolved, and (hidden) unit tests that went from fail to pass after the human-written patch. To resolve an instance, the model is required to generate a patch that can pass these unit tests.

SWE Agents. In this paper, we use the term "SWE agents"[1] to refer to *any* LLM-based system that generates patches to solve issues in a code base, e.g., an instance in SWE-Bench. While the specific implementation varies, a typical SWE agent usually gives their underlying LLM several tools in the form of callable functions to navigate through the code base, find relevant context, edit files, and run tests. The workflow of SWE agents often involves multiple LLM calls, each taking some or all outputs from previous steps as input.

[1]According to our definition, SWE-agent (Yang et al., 2024) is an instance of SWE agents, and Agentless (Xia et al., 2024), despite the name, is another.

3.2 DIVERSITY OF SWE AGENTS

We consider two types of diversity: *intra-agent diversity* and *inter-agent diversity*.

Intra-agent diversity is defined as the degree to which different runs of the same agent solve different problem instances. It is most likely from the non-determinism of the underlying LLM due to sampling in decoding and mixture-of-experts architecture (Chann, 2023). Since the workflow of SWE agents involves multiple steps and LLM calls, a slight difference in an earlier step can easily propagate and result in significant differences in the final outcome.

Inter-agent diversity is defined as the degree to which different agents solve different problem instances. Besides sharing the potential causes of intra-agent diversity, inter-agent diversity is also largely because of differences in agent design, including different tools, workflows, and prompts.

3.3 APPROACH

3.3.1 SWE AGENT PROBLEM FORMULATION

We formulate the SWE agent problem under the *contextual Markov decision process* (CMDP) framework (Hallak et al., 2015), represented by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{C}, \mathcal{A}, \mathcal{R}, \mathcal{P}, p_0, \rho)$. Here, $\mathcal{S}$ denotes the state space, which encompasses all possible states the agent could encounter, such as the current status of files. The context space, $\mathcal{C}$, includes relevant repository information and issue descriptions. The action space, $\mathcal{A}$, represents all potential actions or tools the SWE agent can utilize, such as `search` or `editing`. The context-dependent reward function, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{C} \to \mathbb{R}$, assigns scores based on the actions taken by the agent. For instance, the reward is high if the agent successfully addresses an issue, while it is low if the action results in new bugs in the repository. The context-dependent transition function, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{C} \to \Delta(\mathcal{S})$, defines how the state of the repository or information changes following a specific action. The context-dependent initial state distribution is denoted by $p_0 : \mathcal{C} \to \Delta(\mathcal{S})$, and $\rho \in \Delta(\mathcal{C})$ represents the context distribution.

Given the initial context $c \sim \rho$ and initial state $s_0 \sim p_0(\cdot \mid c)$, at each time step t, the agent follows a policy $\pi : \mathcal{S} \times \mathcal{C} \to \Delta(\mathcal{A})$ to select an action $a_t \sim \pi(s_t, c)$ and receives a reward $\mathcal{R}(s_t, a_t, c)$. The environment then transitions to the next state $s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t, c)$, providing the agent with a new state observation. As the iteration progresses to time T, a sampled trajectory $\tau := \{s_t, a_t, r_t\}_{t=0}^{T}$ is obtained. The objective of an SWE agent is to maximize the cumulative reward along the trajectory, which is captured by the value function:

$$\max_{\pi} V^{\pi}(\rho) = \max_{\pi} \mathbb{E}_{\tau} \left[\sum_{t=0}^{T} \mathcal{R}(s_t, a_t, c) \mid c \sim \rho; \pi \right] \tag{1}$$

3.3.2 OUR FRAMEWORK: DIVERSITY EMPOWERED INTELLIGENCE (DEI)

Many efforts have been made to implement sophisticated agent systems that aim to achieve the objective described in Eq. 1. However, as discussed in Section 1, these systems often exhibit varying levels of effectiveness across different contexts. It is challenging to devise a single agent that can consistently perform well across all possible contexts.

Formally, suppose there are N agent policies, denoted as $\{\pi_1, \pi_2, \ldots, \pi_N\}$, where each policy is tailored to address a specific context $\{\rho_1, \rho_2, \ldots, \rho_N\}$. The union of these contexts is a subset of the entire context space, i.e., $\rho_1 \cup \rho_2 \cup \cdots \cup \rho_N \subseteq \rho$. For each agent policy π_i, the objective is:

$$\pi_i = \max_{\pi} \mathbb{E}_{\tau} \left[\sum_{t=0}^{T} \mathcal{R}(s_t, a_t, c) \mid c \sim \rho_i; \pi \right]. \tag{2}$$

However, an agent policy π_i may perform poorly in a different context ρ_j (where $j \neq i$). To address this limitation, we propose our framework: Diversity Empowered Intelligence (DEI). The DEI framework leverages the strengths of each agent in their respective contexts to enhance overall performance across all contexts.

We introduce a meta-policy, denoted as π_{DEI}, which aims to optimally select among the available agent policies based on the context. The goal of π_{DEI} is defined as:

$$\pi_{\text{DEI}} = \max_{\pi} \mathbb{E}_{c \sim \rho} \left[\mathbb{E}_{\tau} \left[\sum_{t=0}^{T} \mathcal{R}(s_t, a_t, c) \mid c; \pi(c) \right] \right], \tag{3}$$

where $\pi(c)$ denotes the selection of the optimal agent policy from $\{\pi_1, \pi_2, \ldots, \pi_N\}$ based on the observed context c. By dynamically choosing the most suitable agent policy for each context, the DEI framework seeks to maximize the expected cumulative reward across all possible contexts.

3.3.3 DEIBASE: A SIMPLE YET EFFECTIVE IMPLEMENTATION

We present **DEIBASE**, a simple yet powerful implementation of the DEI framework, tailored for SWE-Bench like problems. The context in the setup includes the repository, along with relevant files and issue descriptions. The meta-policy's action space consists of the final patches generated by different agent frameworks, each specialized in addressing various aspects of the problem.

DEIBASE utilizes a Large Language Model (LLM) as a code review committee. The LLM evaluates candidate patches by analyzing the state of the code base before and after the proposed changes, in conjunction with the contextual information from the issue descriptions. It produces detailed explanations for each patch, justifying the modifications based on the identified issues, the context, and the specific changes made.

While other methods of code review and scoring, such as rule-based approaches, can be incorporated into our framework, the use of an LLM-based committee offers a unique advantage. LLMs often excel at evaluating solutions when evaluation is easier than generation. **DEIBASE** thus serves as an effective baseline for LLM-based SWE evaluation, highlighting potential performance variations among diverse SWE agents and showcasing the capabilities of our method.

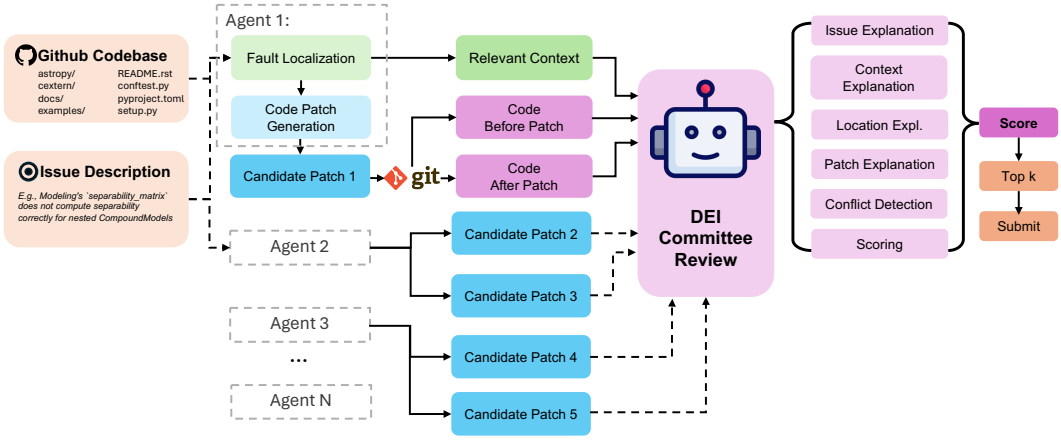

Figure 2: **Framework Overview.** DEI first examines the code base before and after a candidate patch, along with other relevant contexts. Then, it generates an explanation for the issue, the context, and the patch and tries to justify the patch. With its own explanation, it scores the candidate patches and picks the top-scoring ones as more likely to be correct.

As demonstrated in Figure 2, **DEIBASE** is given multiple candidate patches for a single issue. These patches might be from running a single SWE agent multiple times or running multiple SWE agents. **DEIBASE** gives each candidate patch a score and then selects the top-scoring candidates as the patches most likely to work.

Step 1: Input Construction. Four inputs are given to **DEIBASE** for each patch: the issue description itself, relevant context (code snippets identified by an SWE agent as relevant to the issue), code before the patch, and code after the patch. The form of inputs reflects two design choices. First, the entire repository is often too large to fit directly in the context limit of LLMs, so we use the relevant context instead to save token costs and help the model focus. Second, the format of a patch is not the easiest for an LLM to read as it switches back and forth between the pre-change code and

the changed code, so we give the code before and after the patch separately to the model for easier understanding. In practice, we directly use the relevant code spans identified by Moatless Tools, an open-source SWE Agent (Örwall, 2024). There might be potential ways of improving the quality of relevant code spans by making them specific to both the issue and the candidate patch, rather than solely dependent on the issue itself.

Step 2: Explanation Generation. To help the model better "understand" the patch before scoring, we instruct it to generate various explanations regarding the patch in a specified order. The order is decided so that the earlier explanations can also help the later ones. We describe each explanation in the order they are generated here: 1) *Issue explanation* explains what the issue is about and what problem it may be causing. 2) *Context explanation* explains how and why each relevant code span (there might be many of these) is relevant to the issue. 3) *Location explanation* explains if and why the patch is modifying the correct part of the code that's faulty. 4) *Patch explanation* explains if and how the patch is fixing the issue. 5) *Conflict detection* is about checking whether the patch conflicts with other relevant code snippets. We explicitly instruct the model to refer back to the earlier explanations while generating the later ones.

Step 3: Patch Scoring. Based on its own explanations, the model is asked to give the candidate patch a score of 1 to 10. We give the model detailed rubrics of what violations/mistakes lead to higher score deductions and what should only be considered minor violations. For example, if the model finds the modification location to be wrong, it is considered a serious mistake.

4 EXPERIMENTS

We aim to answer two research questions with our experiments: 1) How diverse are LLM-based SWE agents in terms of intra- and inter-agent diversity? 2) To what extent can DEI harness the diversity and increase the performances of these SWE agents?

4.1 EXPERIMENT SETUP

4.1.1 BENCHMARK AND AGENTS

Benchmark. We conduct our experiments on SWE-Bench Lite, a 300-instance subset sampled from the full SWE-Bench for providing a more self-contained evaluation of functional bug fixes (Jimenez et al., 2024). Compared to the full SWE-Bench, SWE-Bench Lite has significantly more submissions on the leaderboard for us to conduct a more comprehensive analysis of inter-agent diversity.

Agents. For intra-agent diversity, we consider three well-performing open-source agents on the SWE-Bench Lite leaderboard: Agentless (Xia et al., 2024), Moatless (Örwall, 2024), and Aider (Gauthier, 2024) by running them 10 times with the same parameters. For inter-agent diversity, we consider 10 agents (open-source or not) that have similar resolve rates, all between 26.0% and 31.0% on the leaderboard by directly using their submitted patches to the SWE-Bench issues. For the evaluation of **DEIBASE** on different agents, we consider 3 groups of agents that form different DEI Committees, including one group consisting of only open-source agents. For the evaluation of **DEIBASE** on multiple runs of a single agent, we use the generations of the three aforementioned agents – Agentless, Moatless Tools, and Aider. More details can be found in Appendix A.1.

4.1.2 EVALUATION METRICS

We use the same set of metrics for both intra- and inter-agent diversity as these metrics are defined for multiple candidate solutions without requiring them to come from the same candidate:

Resolve rate measures how good a SWE agent is. It is defined as the percentage of issues resolved by the agent. We measure both single SWE agents and DEI with it to see how much DEI helps.

Union@k measures the *best case* performance had the agents been perfectly consistent by counting the number of problems solved by any of the k solutions. In the ideal case where the agents are perfectly consistent, Union@k should be the same as Union@1. Union@k can also be considered as the case where we have an oracle reward function $\mathcal{R}^{\text{oracle}}$ that always selects the correct candidate.

Intersect@k measures the *worst case* performance by computing the number of problems solved by all k solutions. The assumption is that a problem is only consistently solved if it's always solved. Yao et al. (2024) calls this metric pass^k. Intersect@k can also be considered as the case where we have an adversarial reward function $\mathcal{R}^{\text{adv}}$ that tries to pick an incorrect candidate if there is one.

Average@k measures the *average case* performance by computing the average number of problems solved. It corresponds to the case of a random reward function $\mathcal{R}^{\text{random}}$ that uniformly samples a candidate solution for each problem.

n@k measures the performance of any reranking mechanism by computing the number of problems solved by n chosen submissions from a total of k samples. The better a reranking mechanism is at telling good solutions from bad ones, the higher n@k is. Note that for an oracle that always picks the correct solution over incorrect ones, n@k is the same as Union@k. For a random reranker that picks a random solution uniformly, n@k is the same as Union@n. In our case, we evaluate $n = 1$.

Our research questions can be answered by the gaps between these metrics. **Union@k - Intersect@** measures how diverse the agents are, while **n@k - Average@k** measures how much DEI helps in selecting the correct candidate from these agents. Note that the order – in which different runs are added – matters as k gets larger, especially when the k candidate solutions come from k different agents. In our experiments, we add candidate solutions from the single agent according to the order they are generated, while we add solutions from different agents in a fixed order (see Appendix A.1).

4.2 Main Results

4.2.1 Research Question 1: How diverse are LLM-based SWE agents?

To answer this question, we report the "@k" metrics of 10 different agents and 10 runs of single agents in Figure 3. The detailed values of these metrics can also be found in Table 2.

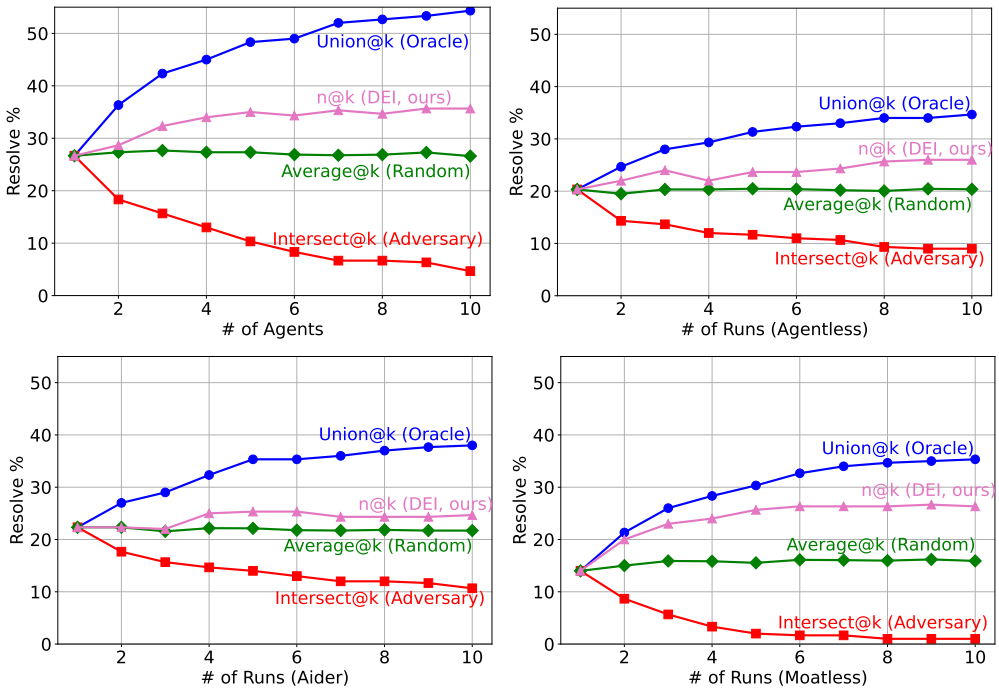

Figure 3: How different metrics change as more candidate solutions are involved. In all 4 scenarios, there is a huge gap between Union@k and Average@k.

Several observations can be made about the results:

SWE agents resolve very different sets of issues across agents and agent runs. Their full potential is far from fully released. In all four subfigures, the gap between Union@k and Average@k, as well as between Average@k and Intersect@k, is large. As k – the number of candidates – gets larger, the gap also gets larger. For 2 of the 4 settings, Union@k is more than 2x larger than Average@k for $k = 10$. The other 2, Union@k is more than 1.5x larger than Average@k for $k = 10$. This indicates that current SWE agents are potentially capable of resolving a lot more issues, as long as we have a reranker that can tell which candidates are correct.

Different agents resolve more distinct issues than different runs of a single agent. In other words, diversity does empower intelligence. The absolute/relative difference between Union@k and Average@k is much larger in the first subfigure than in the following three subfigures. For the "10 different agents" setting, as k approaches 10, the distinct issues resolved are $2\times$ the average number of issues resolved by a single agent in the group.

Table 1: Resolve rates of top-ranking submissions on SWE-Bench Lite. We evaluate 3 DEI Committees formed by different groups of agents. Each DEI Committee outperforms the best agent in it significantly. **DEIBASE**-Open, a committee formed by 4 open-source agents can beat many closed-source agents.

DEI Group	% Resolve	System	Open Src	Trajs	Open Candidates	Backend LLM
1	**55.0**	Salesforce Research **DEIBASE**-1	✓	✓	✗	gpt4o
1	50.6	Cosine Genie	✗	✗	-	"Fine-tuned OpenAI"
1	43.0	CodeStory Aide	✗	✗	-	gpt4o, Claude 3.5 Sonnet
-	38.0	AbenteAI MentatBot	✗	✗	-	gpt4o
2	**37**	Salesforce Research **DEIBASE**-2	✓	✓	✗	gpt4o
Open	**34.3**	Salesforce Research **DEIBASE**-Open	✓	✓	✓	gpt4o
-	34.0	Bytedance MarsCode	✗	✗	-	gpt4o
-	33.0	Alibaba Lingma	✗[1]	✗	-	gpt-4-1106-preview
2	31.3	Factory Code Droid	✗	✗	-	"Anthropic and OpenAI"
2	30.6	AutoCodeRover	✗[2]	✗	-	gpt4o
2	29.6	Amazon Q Dev.	✗	✗	-	Unknown
2	28.3	CodeR	✗[1]	✗	-	gpt-4-1106-preview
2	28.0	MASAI	✗[1]	✗	-	Unknown
-	27.6	SIMA	✗[1]	✓	✓[3]	gpt4o
Open	27.3	Agentless	✓	✓	-	gpt4o
Open	26.6	Moatless Tools	✓	✓	-	Claude 3.5 Sonnet
-	26.6	IBM Research Agent	✗	✗	-	Unknown
Open	26.3	Aider	✓	✗	-	gpt4o, Claude 3 Opus
Open	26.0	OpenDevin + CodeAct	✓	✓	-	gpt4o

[1] Their repo has no code yet.
[2] An earlier version is open-source. The current one is not.
[3] Candidates are generated by a "modification of moatless tools".

4.2.2 RESEARCH QUESTION 2: HOW MUCH DOES DEI HELP?

We apply **DEIBASE** to the candidates in Figure 3 as they are added to the group. Our findings are:

DEIBASE helps in most cases. For most values of k in all subfigures, we observe a significant improvement of n@k over Average@k, indicating that **DEIBASE** selects correct candidates much better than a random baseline.

DEIBASE helps more when the candidates come from different agents. This finding resonates with a similar finding from research question one: Since candidates from multiple agents have a larger potential for improvement (Union@k - Average@k), the actual improvements created by **DEIBASE** (n@k - Average@k) are also larger. This suggests that given a limited budget of candidates, it would be better to choose a diversity of agents over multiple runs of the same agent.

As k gets larger, DEIBASE's improvement first increases and then plateaus. While larger k generally indicates higher n@k, the margin gets smaller and there are cases when an increase in k results in a slight drop in performance. This suggests that the current **DEIBASE** is not ideal for a large group of agents and there is still room for a better reranking mechanism.

Based on the lessons above, we propose three **DEIBASE** groups in which each candidate is from a different agent and no more than 5 candidates exist for each instance. The members of these **DEIBASE** groups and their performance are reported in Table 1. **DEIBASE**-1 consists of the top 2

agents. **DEIBASE**-2 consists of 5 closed-source agents that have high performance on the leaderboard. **DEIBASE**-Open consists of 4 open-source agents so that we know future researchers can run the entire pipeline. As Table 1 shows, all three **DEIBASE** instances outperform the best candidate in the group. Surprisingly, **DEIBASE**-Open shows a 7% increase in resolve rates and beats most of the closed-source systems.

Table 2: How different metrics change as more candidate solutions are involved. As the number of candidates k gets larger, the improvement from DEI also increases significantly.

System	k	Intersect@k	Average@k	1@k (DEI, ours)	Improvement from DEI	Union@k
10 Agents	1	26.7	26.7	26.7	+0.0	26.7
	2	18.3	27.3	28.7	+1.3	36.3
	3	15.7	27.7	32.3	+4.7	42.3
	4	13.0	27.3	34.0	+6.7	45.0
	5	10.3	27.3	35.0	+7.7	48.3
	6	8.3	26.9	34.3	+7.4	49.0
	7	6.7	26.8	35.3	+8.6	52.0
	8	6.7	26.9	34.7	+7.8	52.7
	9	6.3	27.3	35.7	+8.4	53.3
	10	4.7	26.6	35.7	+9.1	54.3
10 Runs from Agentless	1	20.3	20.3	20.3	+0.0	20.3
	2	14.3	19.5	22.0	+2.5	24.7
	3	13.7	20.3	24.0	+3.7	28.0
	4	12.0	20.3	22.0	+1.7	29.3
	5	11.7	20.5	23.7	+3.2	31.3
	6	11.0	20.4	23.7	+3.3	32.3
	7	10.7	20.2	24.3	+4.1	33.0
	8	9.3	20.0	25.7	+5.6	34.0
	9	9.0	20.4	26.0	+5.6	34.0
	10	9.0	20.4	26.0	+5.6	34.7
10 Runs from Aider	1	22.3	22.3	22.3	+0.0	22.3
	2	17.7	22.3	22.3	+0.0	27.0
	3	15.7	21.6	22.0	+0.4	29.0
	4	14.7	22.2	25.0	+2.8	32.3
	5	14.0	22.1	25.3	+3.2	35.3
	6	13.0	21.8	25.3	+3.6	35.3
	7	12.0	21.7	24.3	+2.6	36.0
	8	12.0	21.8	24.3	+2.5	37.0
	9	11.7	21.7	24.3	+2.6	37.7
	10	10.7	21.7	24.7	+3.0	38.0
10 Runs from Moatless	1	14.0	14.0	14.0	+0.0	14.0
	2	8.7	15.0	20.0	+5.0	21.3
	3	5.7	15.9	23.0	+7.1	26.0
	4	3.3	15.8	24.0	+8.2	28.3
	5	2.0	15.5	25.7	+10.1	30.3
	6	1.7	16.1	26.3	+10.2	32.7
	7	1.7	16.0	26.3	+10.3	34.0
	8	1.0	16.0	26.3	+10.4	34.7
	9	1.0	16.2	26.7	+10.5	35.0
	10	1.0	15.9	26.3	+10.4	35.3

4.3 ABLATION AND ANALYSES

In this subsection, we demonstrate some ablation studies to investigate the effectiveness of different components in the framework, in order to answer the following questions. To advocate for open science, all the ablation experiments are conducted on either our own reproduction of open-source SWE agents or their official generations.

Question 1: Does DEI get better with more votes?

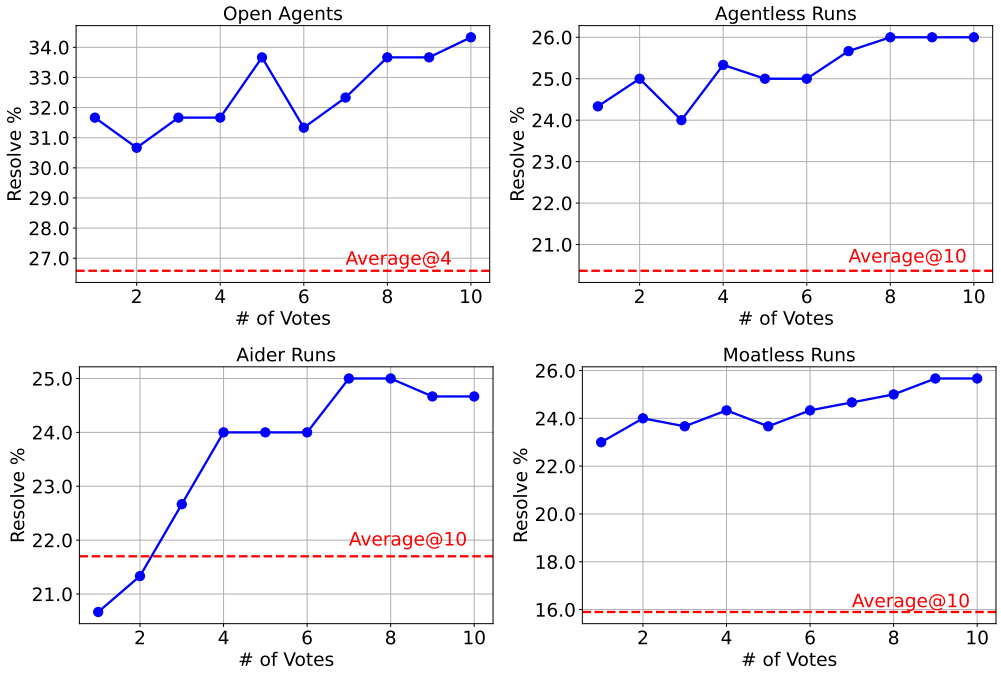

Figure 4: How the performance of **DEIBASE** changes as the LLM is given more votes for scoring.

Answer 1: Yes. Arguably, DEI itself has the same potential characteristics as SWE agents that may cause diverse outputs. So it is important for us to harness the diverse outputs of DEI as well. However, unlike SWE agents whose outputs are code patches, DEI's output is an integer score, which can easily be aggregated and averaged. This is why we give DEI more votes and rerank the candidates according to the average of scores. In most **DEIBASE** experiments, we allow 10 votes for each candidate patch. To investigate whether more votes lead to better patch reviewing, we directly take the scores generated for **DEIBASE**-Open, **DEIBASE**-Agentless, **DEIBASE**-Aider, and **DEIBASE**-Moatless, and check for various values of m, how the first m scores can help us find the best patch.

As demonstrated in Figure 4, more votes generally lead to better resolve rates. Another finding is that for 3 out of the 4 evaluation settings, **DEIBASE** was able to get much better performance than the average candidate with only one vote. Even when **DEIBASE** wasn't able to get better than average with one vote, it managed to get an improvement with only three votes. These results suggest that **DEIBASE** itself also produces diverse outputs, but it is easier to aggregate them via score averaging.

Question 2: Are the explanations necessary?

Answer 2: Yes. We remove the part about asking for explanations from the prompt and compare **DEIBASE**-Open, **DEIBASE**-Agentless, **DEIBASE**-Aider, and **DEIBASE**-Moatless under the same evaluation setting with and without explanations. We report their resolve rates in Table 3. For all 4 settings we evaluated, **DEIBASE** with explanations performs slightly better than **DEIBASE** without explanations.

Table 3: Comparing **DEIBASE**'s resolve rates with and without explanations.

	Open Agents	Agentless	Aider	Moatless
DEIBASE w/ expl.	34.6	26.0	24.6	25.6
DEIBASE w/ o expl.	32.3	23.0	23.3	25.3

5 CONCLUSION

In this paper, we present Diversity Empowered Intelligence (DEI), a meta-policy module designed to integrate with any existing SWE agent frameworks to enable scalable management and collaboration among specialized agents, thereby fostering a more powerful software engineering organization. Through extensive evaluations, we find that different agents show a great level of diversity in the issues they resolve: a group of agents with an average resolve rate of 26.6% can actually solve 54.3% of the issues if we have an oracle that selects the correct candidate. DEI, as our first step towards harnessing such diversity, can improve the group's resolve rate to 34.3% (+7%), suggesting that LLMs are great code reviewers. These findings mirror the benefits of diversity in the tech industry, where diverse perspectives and skills lead to greater innovation and problem-solving capabilities.

Broader Impacts. DEI represents our initial step toward realizing a fully automated organizational AI. We believe that the full potential of multi-agent AI systems extends beyond enhancing task completion accuracy with agentic workflows, which is the current focus of most industry practices. Instead, DEI offers a horizontal, scaling-out approach that facilitates the collaboration and integration of existing diverse agents without necessitating refactoring of engineering work. This capability not only optimizes and speeds up immediate software development tasks but also sets the groundwork for future innovations in AI-driven organizational management.

REFERENCES

Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better llm-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*, 2023.

Seherman Chann. Non-determinism in gpt-4 is caused by sparse moe. *Accessed on August*, 5:2023, 2023.

Dong Chen, Shaoxin Lin, Muhan Zeng, Daoguang Zan, Jian-Gang Wang, Anton Cheshkov, Jun Sun, Hao Yu, Guoliang Dong, Artem Aliev, et al. Coder: Issue resolving with multi-agent and task graphs. *arXiv preprint arXiv:2406.01304*, 2024.

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *arXiv preprint arXiv:2308.10848*, 2023.

Paul Gauthier. Aider - ai for customer service and support, 2024. URL https://aider.chat/. Accessed: 2024-07-16.

Github. Babyagi. *Github — babyagi*, 2023.

Assaf Hallak, Dotan Di Castro, and Shie Mannor. Contextual markov decision processes. *arXiv preprint arXiv:1502.02259*, 2015.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VtmBAGCN7o.

Carlos E. Jimenez, John Yang, and Jiayi Geng. Swe-bench lite: A canonical subset for efficient evaluation of language models as software engineers, March 19 2024. URL https://www.swebench.com/lite.html. Accessed: 2024-07-16.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. Encouraging divergent thinking in large language models through multi-agent debate. *arXiv preprint arXiv:2305.19118*, 2023.

Zhiwei Liu, Weiran Yao, Jianguo Zhang, Le Xue, Shelby Heinecke, Rithesh Murthy, Yihao Feng, Zeyuan Chen, Juan Carlos Niebles, Devansh Arpit, et al. Bolaa: Benchmarking and orchestrating llm-augmented autonomous agents. *arXiv preprint arXiv:2308.05960*, 2023.

Zhiwei Liu, Weiran Yao, Jianguo Zhang, Liangwei Yang, Zuxin Liu, Juntao Tan, Prafulla K Choubey, Tian Lan, Jason Wu, Huan Wang, et al. Agentlite: A lightweight library for building and advancing task-oriented llm agent system. *arXiv preprint arXiv:2402.15538*, 2024.

Yingwei Ma, Qingping Yang, Rongyu Cao, Binhua Li, Fei Huang, and Yongbin Li. How to understand whole software repository? *arXiv preprint arXiv:2406.01422*, 2024.

John Schulman, Barret Zoph, Christina Kim, Jacob Hilton, Jacob Menick, Jiayi Weng, Juan Felipe Ceron Uribe, Liam Fedus, Luke Metz, Michael Pokorny, et al. Introducing chatgpt. *OpenAI Blog*, 2022.

Noah Shinn, Federico Cassano, Beck Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

OpenAI Team. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774.

Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024a.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *ICML*, 2024b.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Opendevin: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024c.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.

Yiran Wu, Tianwei Yue, Shaokun Zhang, Chi Wang, and Qingyun Wu. Stateflow: Enhancing llm task-solving through state-driven workflows. *arXiv preprint arXiv:2403.11322*, 2024.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. Autocoderover: Autonomous program improvement. *arXiv preprint arXiv:2404.05427*, 2024.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jurgen Schmidhuber. Language agents as optimizable graphs. *arXiv preprint arXiv:2402.16823*, 2024.

Albert Örwall. Moatless tools, June 5 2024. URL https://github.com/aorwall/moatless-tools. Accessed: 2024-07-16.

A APPENDIX

A.1 AGENTS EVALUATED

We add the following agents to the DEI Committee (the one in Figure 3) in the order they appear (the order is generated by randomly shuffling their chronological order using python's random shuffle function with a random seed of 42):

1. `20240612 IBM Research Agent101`
2. `20240612 MASAI gpt4o`
3. `20240604 CodeR`
4. `20240523 aider`
5. `20240630 agentless gpt4o`
6. `20240617 moatless gpt4o`
7. `20240725 opendevin codeact v1.8 claude35sonnet`
8. `20240706 sima gpt4o`
9. `20240621 autocoderover-v20240620`
10. `20240509 amazon-q-developer-agent-20240430-dev`